Sprint 4: BigQuery

Jordi Calmet Xartó - ITAAD

https://github.com/jordi-cx/it_academy/tree/main/data_analytics/sprint_04

Nivell 1: Entorn i Ingesta Híbrida (Code-First)

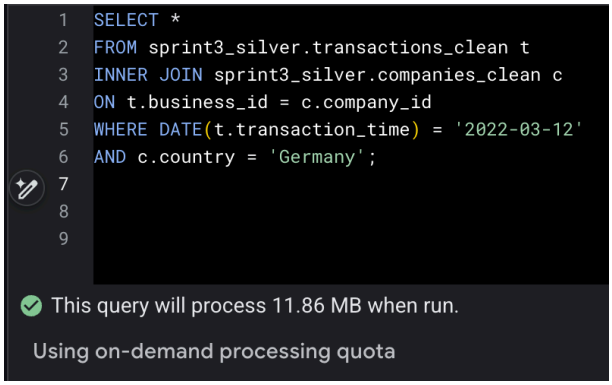
Nivell 1 - Exercici 1:
Consulta sobre Taula no Optimitzada (Diagnòstic)

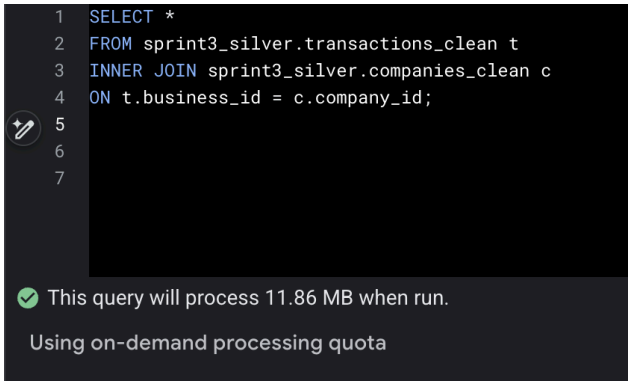
El Country Manager d'Alemanya necessita revisar urgentment les transaccions del dia 12 de març de 2022.
» 1. Escriu la consulta que uneix (JOIN) transaccions i companyies.
» 2. Filtra els resultats per la data indicada i el país "Germany".
» 3. Sense executar la consulta, realitza un "Dry Run" (auditoria de costos).

Voy a intentar realizar la siguiente consulta (pero sin ejecutarla) para ver los costes en el Dry Run:

```
SELECT *
FROM sprint3_silver.transactions_clean t
INNER JOIN sprint3_silver.companies_clean c
ON t.business_id = c.company_id
WHERE DATE(t.transaction_time) = '2022-03-12'
AND c.country = 'Germany';
```

Observamos como esta consulta nos costaría lo mismo tanto si aplicamos el filtro de fecha y país como si no, es decir, haría prácticamente un **full table scan** y ésto no nos interesa, tendríamos que procurar optimizar los costes:





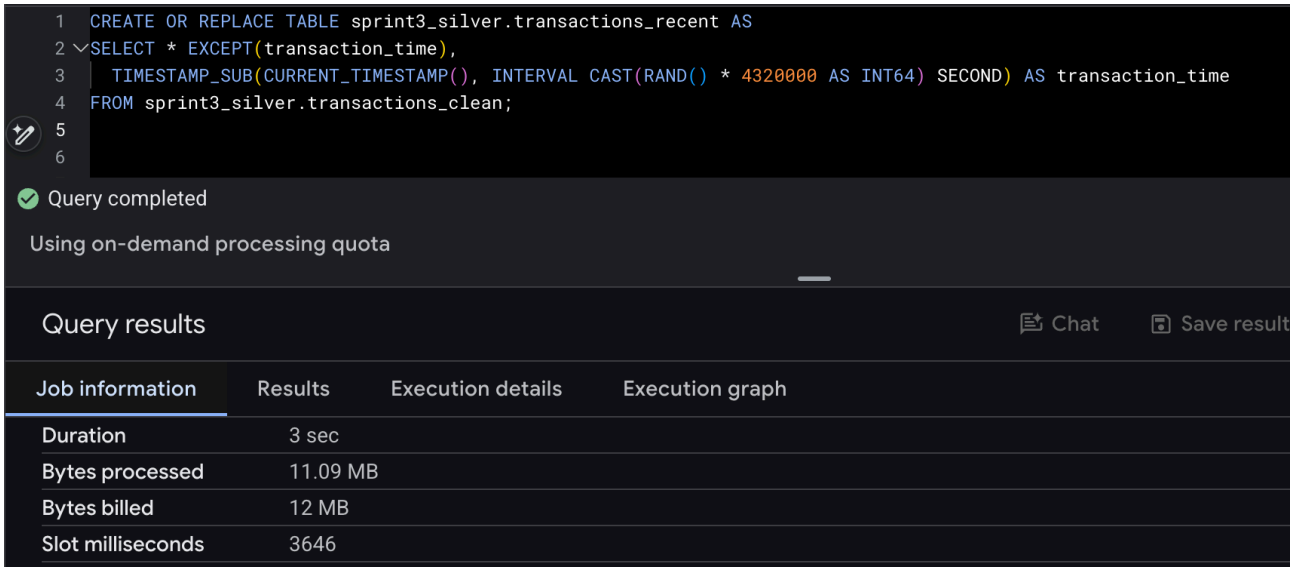
En ambos casos, la consulta procesaría 11.86 MB.

Nivell 1 - Exercici 2:
Re-arquitectura i Optimització de l'Emmagatzematge (Partition & Cluster)

Pas 1: Generació de Dades Recents (Mocking Data)
Crea una taula intermèdia anomenada sprint3_silver.transactions_recent a partir de la taula sprint3_silver.transactions_clean. El teu objectiu és mantenir totes les columnes, però substituir el timestamp original per un de nou, generat aleatòriament perquè caigui dins dels últims 50 dies.

Creo la tabla *transactions_recent* usando un número aleatorio de entre 50 (días) x 24 (horas) x 60 (minutos) x 60 (segundos) = 4320000 segundos restados de la fecha actual, mediante la instrucción:

```
CREATE OR REPLACE TABLE sprint3_silver.transactions_recent AS
SELECT * EXCEPT(transaction_time),
     TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL CAST(RAND() * 4320000
     AS INT64) SECOND) AS transaction_time
FROM sprint3_silver.transactions_clean;
```



Pas 2: Creació de la Taula Optimitzada (Partitioning & Clustering)
Ara, crea la teva taula definitiva sprint3_gold.fact_transactions_optimized a partir de les dades recents que acabes de generar a transactions_recent. Has de construir la sentència DDL per configurar la taula amb les següents estratègies físiques d'emmagatzematge:

» Particionament (Partitioning): Divideix la taula per la data del camp DATE(timestamp). Això permetrà que les consultes que filtrin per dia (WHERE date = ...) només llegeixin la partició necessària.
» Clustering: Ordena les dades dins de cada partició per business_id. Això accelerarà dràsticament els filtres per companyia i els encreuaments (JOINs) amb la dimensió d'empreses.

Ahora voy a crear la nueva tabla optimizada usando las **particiones** y **clústering** que nos piden:

```sql
CREATE OR REPLACE TABLE sprint3_gold.fact_transactions_optimized
PARTITION BY DATE(transaction_time)
CLUSTER BY business_id
AS SELECT *
FROM sprint3_silver.transactions_recent;
```

Query completed

Using on-demand processing quota

Query results

Job information | Results | Execution details | Execution graph

Duration	4 sec
Bytes processed	11.85 MB
Bytes billed	12 MB

Haciendo *Preview* de los datos de esta nueva tabla *fact_transactions_optimized*, podemos ver como efectivamente las filas aparecen agrupadas primero según la **partición** por días (*2026-03-02, …, 2026-04-20*) y luego en orden de **clustering** según el identificador de la compañía (*business_id: b-2222 … b-2610)*:

1993	2DB7CB4A-8ED4-4EC0-B83B-14...	CcU-4583	b-2254	79.53	1	30	40	37.345637...	-121.878...	2026-03-09 21:19:15.768594 UTC
1994	B5A00051-4828-48CA-87F9-47...	CcU-4317	b-2282	49.7	1	11	78	29.421929...	-98.4868...	2026-03-09 18:23:19.768594 UTC
1995	A2F87C7C-C508-438A-A769-DE...	CcU-3351	b-2362	361.72	1	62	216	49.888810...	-97.1459...	2026-03-09 06:17:26.768594 UTC
						89				
1996	3D2869C9-52D8-4903-9666-08...	CcU-4856	b-2382	394.73	1	86	1	40.719221...	-74.0058...	2026-03-09 06:30:30.768594 UTC
						60				
						97				
						4				
1997	E4E7670F-CE7B-469E-865F-B7...	CcU-3239	b-2530	363.14	1	67	232	43.456389...	-80.4866...	2026-03-09 23:37:44.768594 UTC
						29				
1998	D1BB146B-D8DF-45BC-8F94-BD...	CcU-4555	b-2598	228.55	1	78	44	32.720128...	-117.163...	2026-03-09 10:42:07.768594 UTC
						11				
1999	EEFE27E9-6221-448B-B296-660...	CcU-3701	b-2610	244.82	1	4	166	54.878974...	-3.67133...	2026-03-09 18:31:20.768594 UTC
						27				
2000	FD57D8E3-E4BB-448C-8615-BA...	CcS-7580	b-2222	40.43	0	100	2999	39.167785...	-7.85032...	2026-03-10 21:26:29.768594 UTC

Captura con los detalles de la tabla:

Table type	Partitioned
Partitioned by	DAY
Partitioned on field	transaction_time
Partition expiry	60 days
Partition filter	Not required
Clustered by	business_id

Nivell 1 - Exercici 3: La Prova del Cotó (Benchmark)

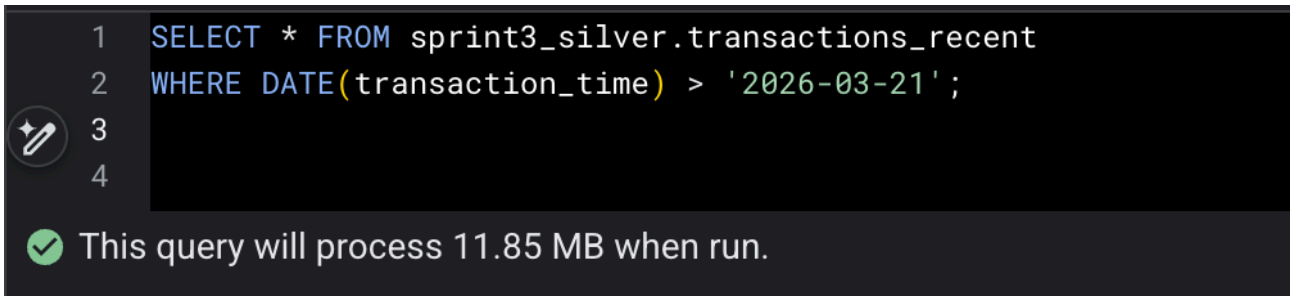
*Construeix una consulta SQL que seleccioni totes les columnes (SELECT *) i filtri les dades dels últims 30 dies.*

» Pas 1 (Taula no optimitzada): Apunta la teva consulta a la taula sprint3_silver.transactions_recent. Observa el validador i fes una captura de pantalla dels bytes que processaria.
» Pas 2 (Taula optimitzada): Sense canviar res de la lògica de la teva consulta, modifica només el FROM perquè ara apunti a la taula particionada sprint3_gold.fact_transactions_optimized. Observa com canvia el validador i fes la segona captura.

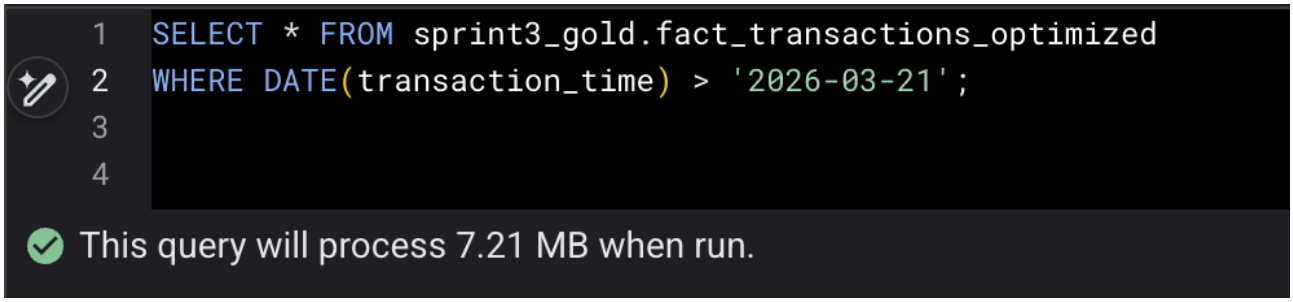
Entrega:
» El codi SQL base que has dissenyat.
» Les dues captures de pantalla on es vegi la diferència de "Bytes processats" al validador.

Veamos qué pasaría al ejecutar exactamente la misma consulta en las dos tablas de transacciones (una sin optimizar y la otra optimizada con particiones y clustering), usando este código:

```
SELECT * FROM sprint3_silver.transactions_recent
WHERE DATE(transaction_time) > '2026-03-21';
```




```
SELECT * FROM sprint3_gold.fact_transactions_optimized
WHERE DATE(transaction_time) > '2026-03-21';
```



» Un breu text calculant quin és el % d'estalvi que has aconseguit (compara els bytes del Pas 1 respecte als del Pas 2).

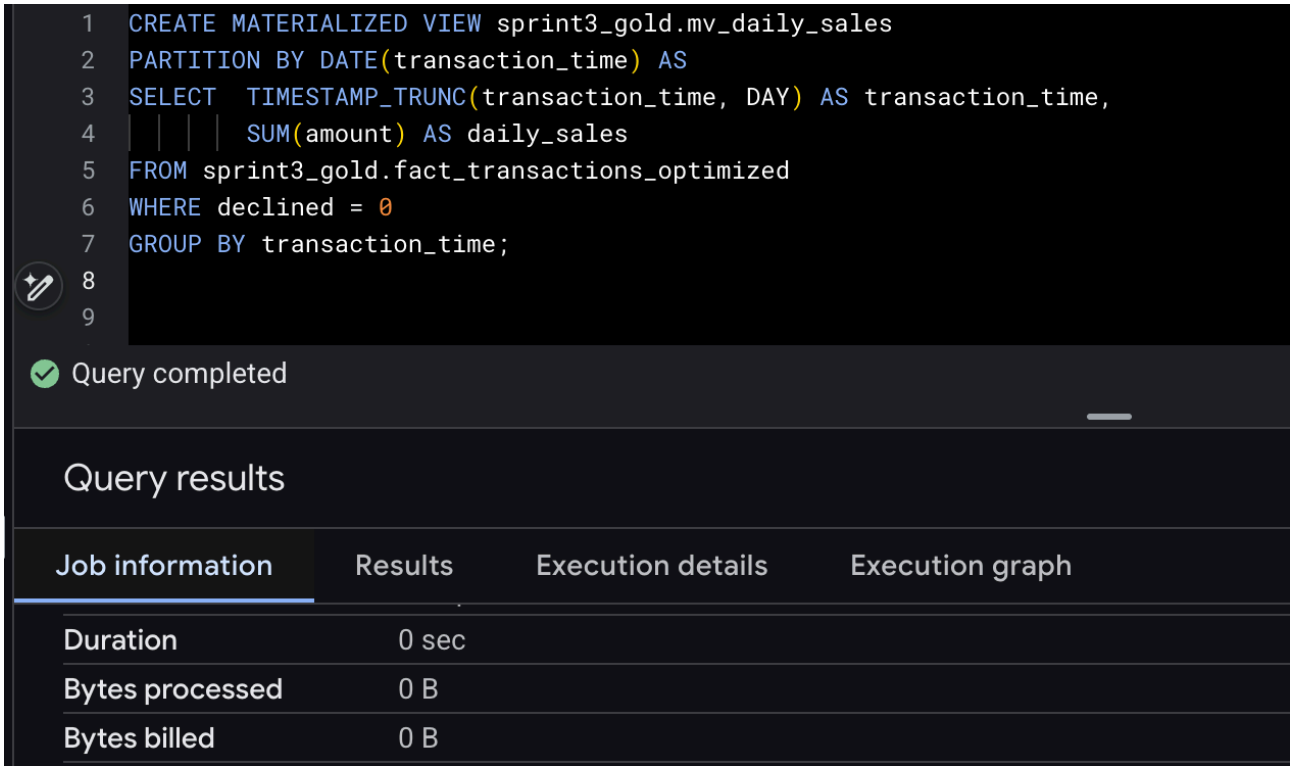
Obervamos pues que el hecho de optimizar la tabla supondría un **ahorro** en el coste de los bytes procesados de aproximadamente un 40%: (7.21 MB - 11.85 MB) * 100 / 11.85 = **-39.15%**

Nivell 1 - Exercici 4: Smart Caching (Vistes Materialitzades)

» Crea una Vista Materialitzada anomenada sprint3_gold.mv_daily_sales que mostri les Vendes Totals per Dia.

```
CREATE MATERIALIZED VIEW sprint3_gold.mv_daily_sales
PARTITION BY DATE(transaction_time) AS
SELECT     DATE(timestamp) AS transaction_time,
           SUM(amount) AS daily_sales
FROM sprint3_gold.fact_transactions_optimized
WHERE declined = 0
GROUP BY transaction_time;
```

» Fes una consulta que mostri la vista creada i fes una captura de pantalla que mostri en el seu conjunt la vista materialitzada creada, amb la consulta executada i els bits processats.



Con la **vista materializada** anterior, ejecutamos la siguiente consulta para ver su rendimiento:

```
SELECT     DATE(transaction_time) AS day,
           ROUND(daily_sales, 2) AS total_sales
FROM sprint3_gold.mv_daily_sales
ORDER BY transaction_time;
```

Podemos comprobar que esta consulta se ha ejectuado de forma muy rápida y con un coste bajo de bytes procesados, y nos muestra los resultados agrupados tal como esperábamos:

```sql
SELECT  DATE(transaction_time) AS day,
        ROUND(daily_sales, 2) AS total_sales
FROM sprint3_gold.mv_daily_sales
ORDER BY day;
```

Query completed

Using on-demand processing quota

Query results

Job information | Results | Visualisation | JSON | Execution details

Duration	0 sec
Bytes processed	816 B
Bytes billed	10 MB
Slot milliseconds	1776

Así podemos ver las ventas ordenadas por días a partir de la vista materializada:

Query results

Job information | Results | Visualisation | JSON

Row	day	total_sales
1	2026-03-02	313976.03
2	2026-03-03	513563.74
3	2026-03-04	497318.15
4	2026-03-05	511793.8
5	2026-03-06	538131.73
6	2026-03-07	510021.11
7	2026-03-08	515120.03
8	2026-03-09	519422.58
9	2026-03-10	518096.71
10	2026-03-11	523351.48
11	2026-03-12	523251.63
12	2026-03-13	527803.61
13	2026-03-14	534604.09
14	2026-03-15	501097.52
15	2026-03-16	513198.16

Nivell 2: SQL Analític Avançat

Nivell 2 - Exercici 1:
Perfilat de Clients VIP (Mètriques Agregades amb CTEs)

» Crea una CTE anomenada VIP_Stats que agrupi per usuari i calculi:
La Despesa Total (SUM).
La Quantitat de Transaccions (COUNT).
El Tiquet Mitjà (AVG), arrodonit a 2 decimals.
La Compra Màxima (MAX).
Filtre: Mantén només aquells la Despesa Total dels quals sigui > 500.

» Creua la CTE amb users_combined per obtenir les dades personals.
Columnes: user_id, nom_complet, email, num_compres, tiquet_mig, max_compra, total_gastat. Ordenat per total_gastat descendent.

Utilizaré la siguiente CTE con toda la información que nos piden:

```
WITH vip_stats AS (
    SELECT  user_id,
            COUNT(transaction_id) AS num_compras,
            ROUND(AVG(amount), 2) AS ticket_medio,
            ROUND(MAX(amount), 2) AS max_compra,
            ROUND(SUM(amount), 2) AS total_gastado
    FROM sprint3_gold.fact_transactions_optimized
    GROUP BY user_id
    HAVING SUM(amount) > 500
)
SELECT  v.user_id,
        CONCAT(u.name, ' ', u.surname) AS nombre_completo,
        u.email,
        v.num_compras,
        v.ticket_medio,
        v.max_compra,
        v.total_gastado
FROM vip_stats AS v
LEFT JOIN sprint3_silver.users_combined AS u USING (user_id)
ORDER BY v.total_gastado DESC;
```

Row	user_id	full_name	e_mail	num_sales	avg_sale	max_sale	total_spent
1	289	Dxwgi Hwcru	dxwgi.hwcru@example.com	91	446.91	554.71	40669.1599999
2	318	Bnyr Astuw	bnyr.astuw@example.com	86	449.0	615.51	38614.3899999
3	455	Fwfll Phylgnmhnd	fwfll.phylgnmhnd@example.com	66	522.59	724.44	34491.2700000
4	417	Fqatil Rdwcjcvoa	fqatil.rdwcjcvoa@example.com	73	421.87	675.78	30796.3000000
5	185	Molly Gilliam	donec@outlook.couk	107	274.72	659.31	29394.7899999
6	285	Brcxprnj Lnpna	brcxprnj.lnpna@example.com	70	417.85	630.52	29249.5100000
7	193	Minerva Wilkins	aliquam.adipiscing@yahoo.edu	67	380.37	646.0	25485.0500000
8	454	Sfzzoh Xgvfridxs	sfzzoh.xgvfridxs@example.com	78	304.0	604.9	23711.9000000
9	349	Yxxcjk Fnceqbkxmt	yxxcjk.fnceqbkxmt@example.c...	53	443.48	675.05	23504.4699999
10	317	Qikcvc Nvfcifqdh	qikcvc.nvfcifqdh@example.com	54	351.69	600.77	18991.4499999
11	388	Rbjupnac Cfjykdx	rbjupnac.cfjykdx@example.com	62	297.3	479.03	18432.8399999
12	319	Fxtg Eedgl	fxtg.eedgl@example.com	62	286.62	576.22	17770.3899999
13	443	Qimzjbfg Cvpdvtz	qimzjbfg.cvpdvtz@example.com	75	230.81	524.33	17310.4199999
14	393	Nxbq Bmbzxiihu	nxbq.bmbzxiihu@example.com	76	220.09	534.22	16726.4699999
15	437	Kxnl Rzhpnaz	kxnl.rzhpnaz@example.com	50	325.21	572.37	16260.4600000
16	183	Germane Whitehead	malesuada.malesuada.integer...	76	194.98	624.64	14818.1700000
17	402	Fpymtvae Kvuvznta	fpymtvae.kvuvznta@example.c...	76	188.35	737.25	14314.2999999
18	120	Solomon Blake	in.at.pede@hotmail.edu	45	318.0	626.48	14310.

```sql
WITH vip_stats AS (
SELECT  user_id,
    CONCAT(u.name, ' ', u.surname) AS full_name,
    u.email AS e_mail,
    COUNT(t.transaction_id) AS num_sales,
    ROUND(AVG(t.amount), 2) AS avg_sale,
    MAX(t.amount) AS max_sale,
    SUM(t.amount) AS total_spent
FROM sprint3_silver.transactions_recent t
INNER JOIN sprint3_silver.users_combined u
USING (user_id)
WHERE t.declined = 0
GROUP BY user_id, full_name, e_mail
HAVING total_spent > 500.00
)
SELECT * FROM vip_stats
ORDER BY total_spent DESC;
```

Query completed

Using on-demand processing quota

Query results

Job information | Results | Visualisation | JSON | Execution details

Duration	0 sec
Bytes processed	6.17 MB
Bytes billed	20 MB
Slot milliseconds	102

Nivell 2 - Exercici 2:
Anàlisi de Tendències (Window Functions sobre Vistes)

Crea una consulta sobre la vista materialitzada que generi un informe amb les següents 4 columnes:
» 1.Data: La data de venda.
» 2.Vendes_Avui: El total venut aquell dia.
» 3.Vendes_Ahir: El total venut el dia anterior (usant funcions de finestra).
» 4.Diff_Percentual: El percentatge de creixement o decreixement respecte al dia anterior, arrodonit a 2 decimals.

Usaré tanto una CTE como una Window Function para componer la consulta que queremos:

```
WITH cte_daily AS (
SELECT
     DATE(transaction_time) AS sales_date,
     ROUND(daily_sales, 2) AS sold_this_day,
     ROUND(LAG(daily_sales) OVER(ORDER BY DATE(transaction_time) ASC),
     2) AS sold_day_before
FROM
     sprint3_gold.mv_daily_sales
)
SELECT     sales_date,
           sold_this_day,
           sold_day_before,
           ROUND(SAFE_DIVIDE((sold_this_day - sold_day_before),
           sold_day_before) * 100, 2) AS percent_diff
FROM cte_daily
ORDER BY sales_date ASC;
```

```
WITH cte_daily AS (
  SELECT
        DATE(transaction_time) AS sales_date,
        ROUND(daily_sales, 2) AS sold_this_day,
        ROUND(LAG(daily_sales) OVER(ORDER BY DATE(transaction_time) ASC), 2) AS sold_day_before
    FROM
        sprint3_gold.mv_daily_sales
)
SELECT
    sales_date,
    sold_this_day,
    sold_day_before,
    ROUND(SAFE_DIVIDE((sold_this_day - sold_day_before), sold_day_before) * 100, 2) AS percent_diff
FROM
    cte_daily
ORDER BY
    sales_date ASC;
```

Query completed

Using on-demand processing quota

Query results

Chat Save results

Job information | Results | Visualisation | JSON | Execution details | Execution graph

Row	sales_date	sold_this_day	sold_day_before	percent_diff
1	2026-03-02	313976.03	null	null
2	2026-03-03	513563.74	313976.03	63.57
3	2026-03-04	497318.15	513563.74	-3.16
4	2026-03-05	511793.8	497318.15	2.91
5	2026-03-06	538131.73	511793.8	5.15
6	2026-03-07	510021.11	538131.73	-5.22

Nivell 2 - Exercici 3:
Totals Acumulats (Running Totals sobre Vistes)

Utilitzant la vista materialitzada mv_daily_sales (per no recalcular agregacions base), genera un informe amb tres columnes:
» 1.Data.
» 2.Vendes del Dia: Arrodonit a 2 decimals.
» 3.Vendes Acumulades YTD: Una suma progressiva de les vendes que ha de reiniciar-se cada 1 de Gener. El resultat final ha d'estar arrodonit a 2 decimals.

Usaré una Window Function para calcular el acumulado anual:

```
SELECT  sales_date AS fecha,
        ROUND(daily_sales, 2) AS ventas_dia,
        ROUND(SUM(daily_sales)
            OVER (PARTITION BY EXTRACT(YEAR FROM sales_date)
            ORDER BY sales_date ROWS BETWEEN UNBOUNDED PRECEDING AND
CURRENT ROW), 2)
            AS ventas_acumuladas_ytd
FROM sprint3_gold.mv_daily_sales
ORDER BY sales_date;
```

```
WITH cte_ytd AS (
  SELECT  DATE(transaction_time) AS sales_date,
          ROUND(daily_sales, 2) AS sold_this_day
    FROM sprint3_gold.mv_daily_sales
)
SELECT  sales_date,
    sold_this_day,
    ROUND(SUM(sold_this_day) OVER (
        PARTITION BY EXTRACT(YEAR FROM sales_date)
        ORDER BY sales_date ASC
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW), 2) AS ytd_sales
FROM cte_ytd
ORDER BY sales_date ASC;
```

This query will process 816 B when run.

Puedo ver que funciona correctamente y obtengo los siguientes resultados:

Job information | Results | Visualisation | JSON

Row	sales_date	sold_this_day	ytd_sales
1	2026-03-02	313976.03	313976.03
2	2026-03-03	513563.74	827539.77
3	2026-03-04	497318.15	1324857.92
4	2026-03-05	511793.8	1836651.72
5	2026-03-06	538131.73	2374783.45
6	2026-03-07	510021.11	2884804.56
7	2026-03-08	515120.03	3399924.59
8	2026-03-09	519422.58	3919347.17
9	2026-03-10	518096.71	4437443.88
10	2026-03-11	523351.48	4960795.36
11	2026-03-12	523251.63	5484046.99
12	2026-03-13	527803.61	6011850.6
13	2026-03-14	534604.09	6546454.69
14	2026-03-15	501097.52	7047552.21
15	2026-03-16	513198.16	7560750.37
16	2026-03-17	511402.82	8072153.19
17	2026-03-18	529159.86	8601313.05
18	2026-03-19	528469.96	9129783.01

Nivell 2 - Exercici 4:
Fidelització i Valor del Client (Filtratge Avançat)

Genera un llistat dels usuaris que han arribat (o superat) la seva tercera compra, mostrant:
» 1. Dades d'usuari (user_id, nom_complet, email).
» 2. La data i l'import exacte de la 3a compra.
» 3. Mitjana de les 3 primeres: La mitjana de despesa de les seves transaccions 1, 2 i 3.

```
WITH ranked AS (
    SELECT  user_id,
            transaction_time,
            amount,
            ROW_NUMBER() OVER(PARTITION BY user_id ORDER BY
transaction_time) AS rn
    FROM sprint3_gold.fact_transactions_optimized
    QUALIFY rn <= 3

),

per_user AS (
    SELECT  user_id,
            ROUND(AVG(amount), 2) AS media_tres_primeras,
            MAX(IF(rn = 3), transaction_time, NULL) AS
fecha_tercera_compra,
            MAX(IF(rn = 3), amount, NULL) AS monto_tercera_compra
    FROM ranked
    GROUP BY user_id
)

SELECT  p.user_id,
        CONCAT(u.name, ' ', u.surname) AS nombre_completo,
        u.email,
        p.fecha_tercera_compra,
        p.monto_tercera_compra,
        p.media_tres_primeras
FROM per_user p
LEFT JOIN sprint3_silver.users_combined u USING (user_id)
WHERE 1 = 1
AND p.fecha_tercera_compra IS NOT NULL
ORDER BY user_id ASC;
```

```sql
WITH first_three_sales AS (
    SELECT
        u.user_id,
        CONCAT(u.name, ' ', u.surname) AS full_name,
        u.email AS e_mail,
        t.amount,
        t.transaction_time,
        ROW_NUMBER() OVER(PARTITION BY user_id ORDER BY transaction_time ASC) AS sale_rank
FROM sprint3_silver.transactions_clean t
INNER JOIN sprint3_silver.users_combined u
USING (user_id)
WHERE t.declined = 0
QUALIFY sale_rank <= 3
)
SELECT
    user_id,
    full_name,
    e_mail,
    MAX(CASE WHEN sale_rank = 3 THEN DATE(transaction_time) END) AS third_sale_date,
    MAX(CASE WHEN sale_rank = 3 THEN amount END) AS third_sale_amount,
    ROUND(AVG(amount), 2) AS avg_three_sales
FROM
    first_three_sales
GROUP BY
    user_id, full_name, e_mail
HAVING
    COUNT(*) = 3
ORDER BY
    avg_three_sales DESC;
```

This query will process 3.31 MB when run.

Query results

Chat | Save results | Open in

Job information | Results | Visualisation | JSON | Execution details | Execution graph

Row	user_id	full_name	e_mail	third_sale_date	third_sale_amount	avg_three_sales
1	444	Eklf Srqdlsxkk	eklf.srqdlsxkk@example.com	2016-05-12	674.66	581.87
2	2124	Ltbjxm Ziielgna	ltbjxm.ziielgna@example.com	2015-08-08	509.37	565.5
3	2762	Mykbqx Iodwcgbk	mykbqx.iodwcgbk@example.com	2017-06-29	695.08	545.52
4	3714	Vfhdkr Vfazkeju	vfhdkr.vfazkeju@example.com	2015-12-15	685.36	525.5
5	300	Aiet Jwcmzsbs	aiet.jwcmzsbs@example.com	2015-11-26	444.5	523.76
6	235	Medge Nieves	ac@icloud.couk	2015-04-28	499.17	522.82
7	3338	Hlyhem Awgcslqd	hlyhem.awgcslqd@example.com	2016-03-15	462.2	521.86
8	479	Qirpsrzg Canoxoykp	qirpsrzg.canoxoykp@example.c...	2015-07-20	301.34	517.92
9	456	Ysgrczww Xrfcht	ysgrczww.xrfcht@example.com	2015-07-10	561.31	517.54
10	3302	Wvnsow Yfbnpmru	wvnsow.yfbnpmru@example.co...	2017-10-14	426.37	514.93
11	3258	Dxcvsz Fqycuoma	dxcvsz.fqycuoma@example.com	2015-04-05	452.03	513.59
12	3100	Wpdeav Rkcojarx	wpdeav.rkcojarx@example.com	2015-07-07	546.52	511.75
13	325	Etwfurw Vfapklpr	etwfurw.vfapklpr@example.com	2016-08-28	597.98	511.38
14	1952	Drbbrl Mwbvcqyd	drbbrl.mwbvcqyd@example.com	2019-02-27	433.56	509.09
15	2858	Zrkgim Wrnjhoor	zrkgim.wrnjhoor@example.com	2017-05-08	582.11	507.95

Nivell 3: Analytics Engineering (Arrays & Automatització)

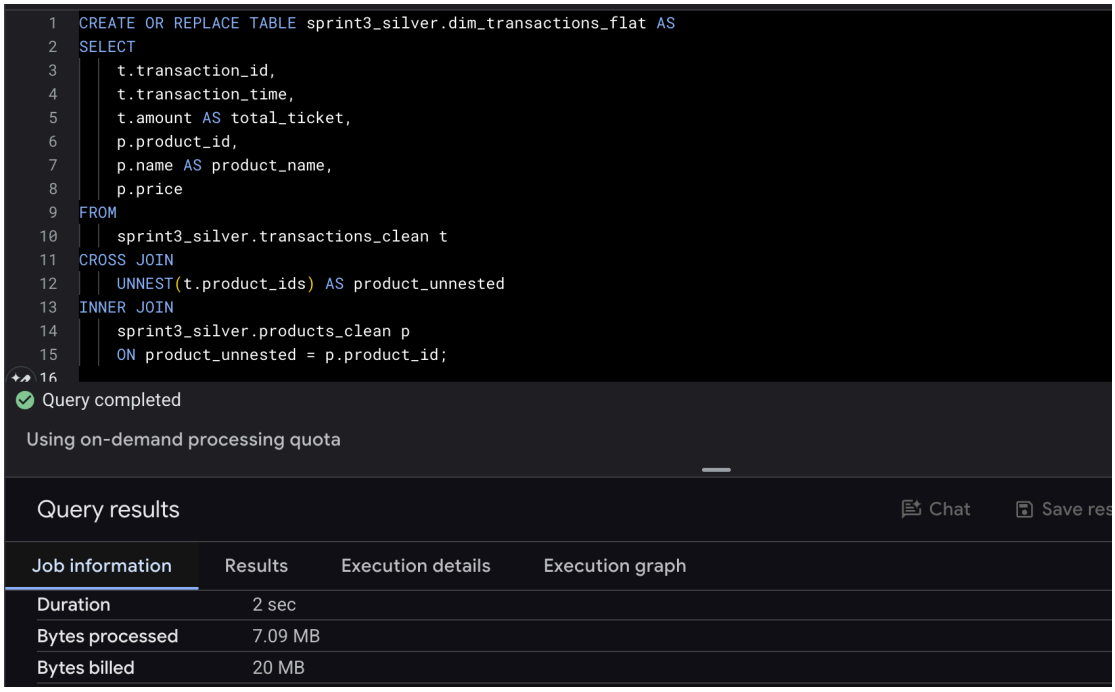
Nivell 3 - Exercici 1:
Desanidament i Aplanament de Dades (Unnesting)

Crea la taula dim_transactions_flat desnormalitzant la informació. Has d'"explotar" l'array de productes i creuar-lo amb el catàleg mestre per obtenir els noms i preus individuals.

Passos Tècnics:
» 1. Utilitza CROSS JOIN UNNEST(product_ids) per transformar l'Array en files individuals.
» 2. Realitza un JOIN amb la taula products per obtenir el nom (name) i preu (price) de cada ítem. Nota: Ves amb compte amb els tipus de dades en fer el JOIN (Array de INT64 vs Product_ID STRING).
» 3. No agrupeu el resultat. Volem veure el desglossament línia per línia.

Voy a crear la nueva tabla aplanada (**flat table**) con la información que nos piden de cada transacción individual y los datos de cada producto:

```
CREATE OR REPLACE TABLE sprint3_silver.dim_transactions_flat AS
SELECT
    t.transaction_id,
    t.transaction_time,
    t.amount AS total_ticket,
    p.product_id,
    p.name AS product_name,
    p.price
FROM
    sprint3_silver.transactions_clean t
CROSS JOIN
    UNNEST(t.product_ids) AS product_unnested
INNER JOIN
    sprint3_silver.products_clean p
    ON product_unnested = p.product_id
WHERE t.declined = 0;
```



(Hago notar que según la tabla que yo tenía de *products_clean,* el *product_id* ya era de tipo INT y por tanto he podido hacer el JOIN directamente sin problema. También he decidido usar el filtro de *declined = 0*, puesto que se trata de una tabla con los productos vendidos para el Equipo de Ventas, creo que interesa tener ya esta tabla con las ventas confirmadas.)

Aquí podemos ver el esquema de la nueva tabla y una muestra de los datos visualizados:

Field name	Type	Mode	Description
transaction_id	STRING	NULLABLE	-
transaction_time	TIMESTAMP	NULLABLE	-
total_ticket	FLOAT	NULLABLE	-
product_id	INTEGER	NULLABLE	-
product_name	STRING	NULLABLE	-
price	FLOAT	NULLABLE	-

```
SELECT * FROM sprint3_silver.dim_transactions_flat
ORDER BY transaction_id;
```

Row	transaction_id	transaction_time	total_ticket	product_id	product_name	price
1	00043A49-2949-494B-A5DD-A5BAE3BB19DD	2024-08-28 07:16:46 UTC	395.43	26	Stark Karstark	53.01
2	00043A49-2949-494B-A5DD-A5BAE3BB19DD	2024-08-28 07:16:46 UTC	395.43	16	the duel warden	180.91
3	00043A49-2949-494B-A5DD-A5BAE3BB19DD	2024-08-28 07:16:46 UTC	395.43	97	jinn Winterfell	65.25
4	00043A49-2949-494B-A5DD-A5BAE3BB19DD	2024-08-28 07:16:46 UTC	395.43	87	sith Jade	96.26
5	000447FE-B650-4DCF-85DE-C7ED0EE1CAAD	2016-12-21 20:07:18 UTC	155.63	66	mustafar jinn	2.12
6	000447FE-B650-4DCF-85DE-C7ED0EE1CAAD	2016-12-21 20:07:18 UTC	155.63	87	sith Jade	96.26
7	000447FE-B650-4DCF-85DE-C7ED0EE1CAAD	2016-12-21 20:07:18 UTC	155.63	69	Littlefinger the giantsblood	57.25
8	00045D6B-ED2E-4F2F-8186-CEE074D875D0	2020-07-14 15:37:45 UTC	326.01	16	the duel warden	180.91
9	00045D6B-ED2E-4F2F-8186-CEE074D875D0	2020-07-14 15:37:45 UTC	326.01	11	Karstark Dorne	49.7
10	00045D6B-ED2E-4F2F-8186-CEE074D875D0	2020-07-14 15:37:45 UTC	326.01	30	Karstark warden	79.53
11	00045D6B-ED2E-4F2F-8186-CEE074D875D0	2020-07-14 15:37:45 UTC	326.01	81	the duel	15.87
12	000481C3-1C26-4FEF-83A0-4CD0EB004BBD	2017-09-04 19:44:53 UTC	161.6	72	north	161.6

Nivell 3 - Exercici 2: El Rànquing de Vendes (Agregació Simple)

Genera el Top 5 de productes més venuts en la història de la companyia.
Consulta directament la teva nova taula desnormalitzada (dim_transactions_flat).

A partir de la tabla anterior, ya podemos realizar fácilmente la consulta que nos piden con el top 5 de los productos vendidos:

```
SELECT product_name, COUNT(*) AS product_sales
FROM sprint3_silver.dim_transactions_flat
GROUP BY product_name
ORDER BY product_sales DESC
LIMIT 5;
```

```
1  SELECT product_name, COUNT(*) AS product_sales
2  FROM sprint3_silver.dim_transactions_flat
3  GROUP BY product_name
4  ORDER BY product_sales DESC
5  LIMIT 5;
6
```

This query will process 4.04 MB when run.

Y vemos por pantalla los siguientes resultados:

Row	product_name	product_sales
1	Winterfell	10041
2	the duel	10034
3	dooku solo	10005
4	Karstark Dorne	7666
5	skywalker ewok	7582

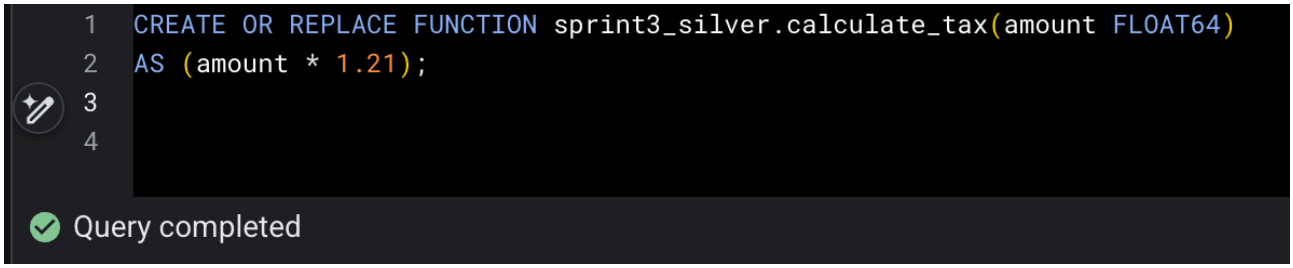
Nivell 3 - Exercici 3: Automatització del Pipeline i Visualització

Has d'evolucionar la teva taula dim_transactions_flat per incloure el càlcul d'impostos i automatitzar la seva regeneració.

1. User Defined Functions (UDF): Crea una funció SQL persistent anomenada calculate_tax(amount) que rebi un valor numèric i retorni el resultat aplicant el 21% d'impost.

» Codi SQL de la UDF:

```
CREATE OR REPLACE FUNCTION sprint3_silver.calculate_tax(amount FLOAT64)
AS (amount * 1.21);
```

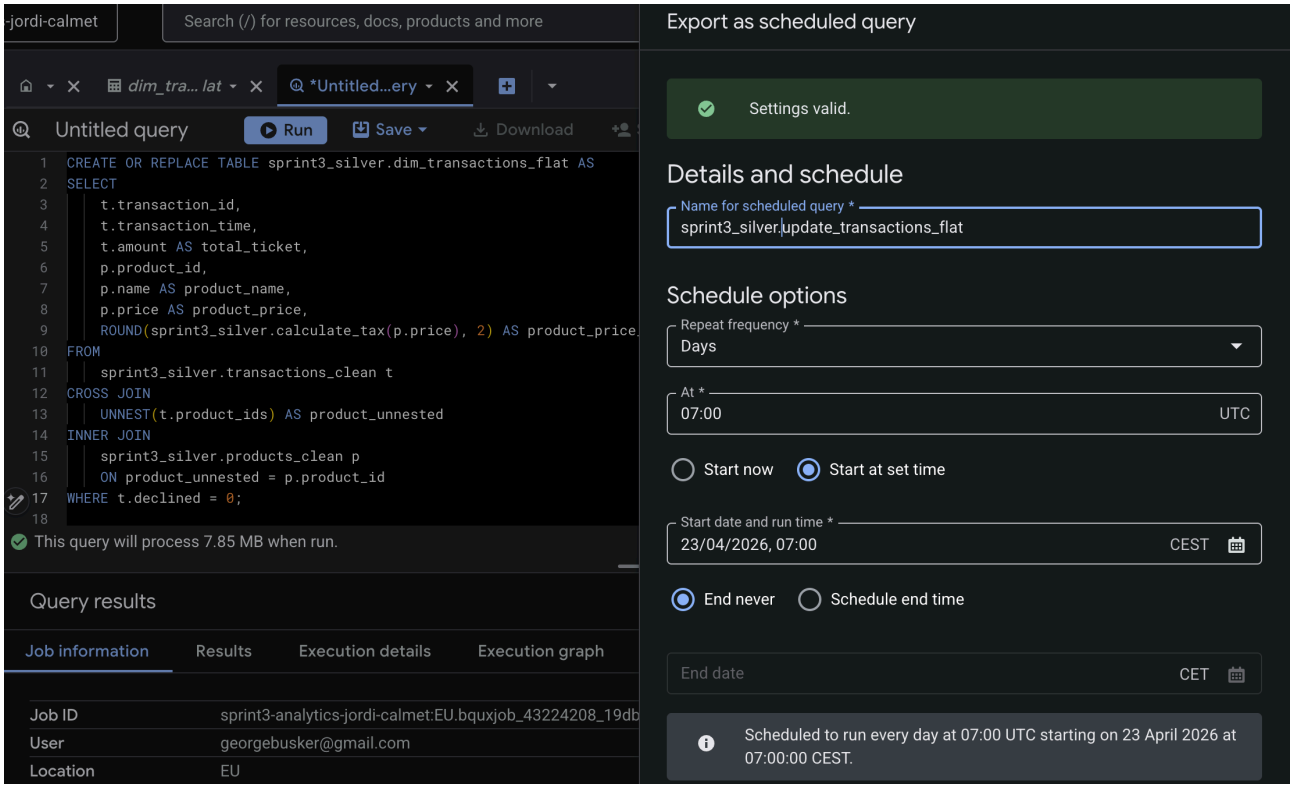


2. Integració i Orquestració: Modifica el codi de creació de la taula (Ex. 1) perquè utilitzi la teva nova funció calculate_tax i generi una columna nova: product_price_tax_inc. Configura una Scheduled Query a BigQuery perquè aquesta sentència (CTAS) s'executi automàticament cada dia a les 07:00 AM.

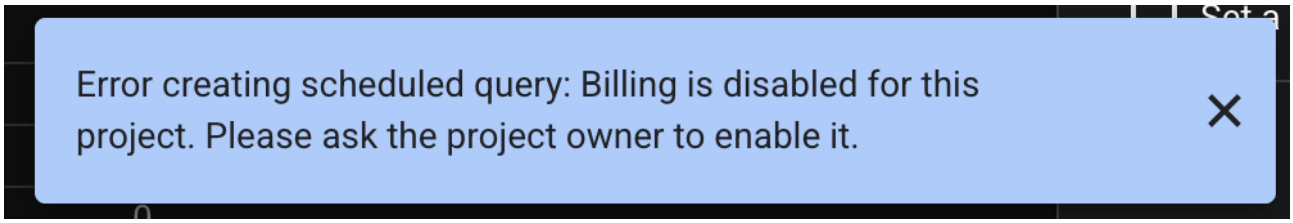
» Codi SQL actualitzat de la creació de la taula:

```
CREATE OR REPLACE TABLE sprint3_silver.dim_transactions_flat AS
SELECT
    t.transaction_id,
    t.transaction_time,
    t.amount AS total_ticket,
    p.product_id,
    p.name AS product_name,
    p.price AS product_price,
    ROUND(sprint3_silver.calculate_tax(p.price), 2)
     AS product_price_tax_inc
FROM
    sprint3_silver.transactions_clean t
CROSS JOIN
    UNNEST(t.product_ids) AS product_unnested
INNER JOIN
    sprint3_silver.products_clean p
    ON product_unnested = p.product_id
WHERE t.declined = 0;
```

» *Una captura de pantalla demostrant que la Scheduled Query està programada:*



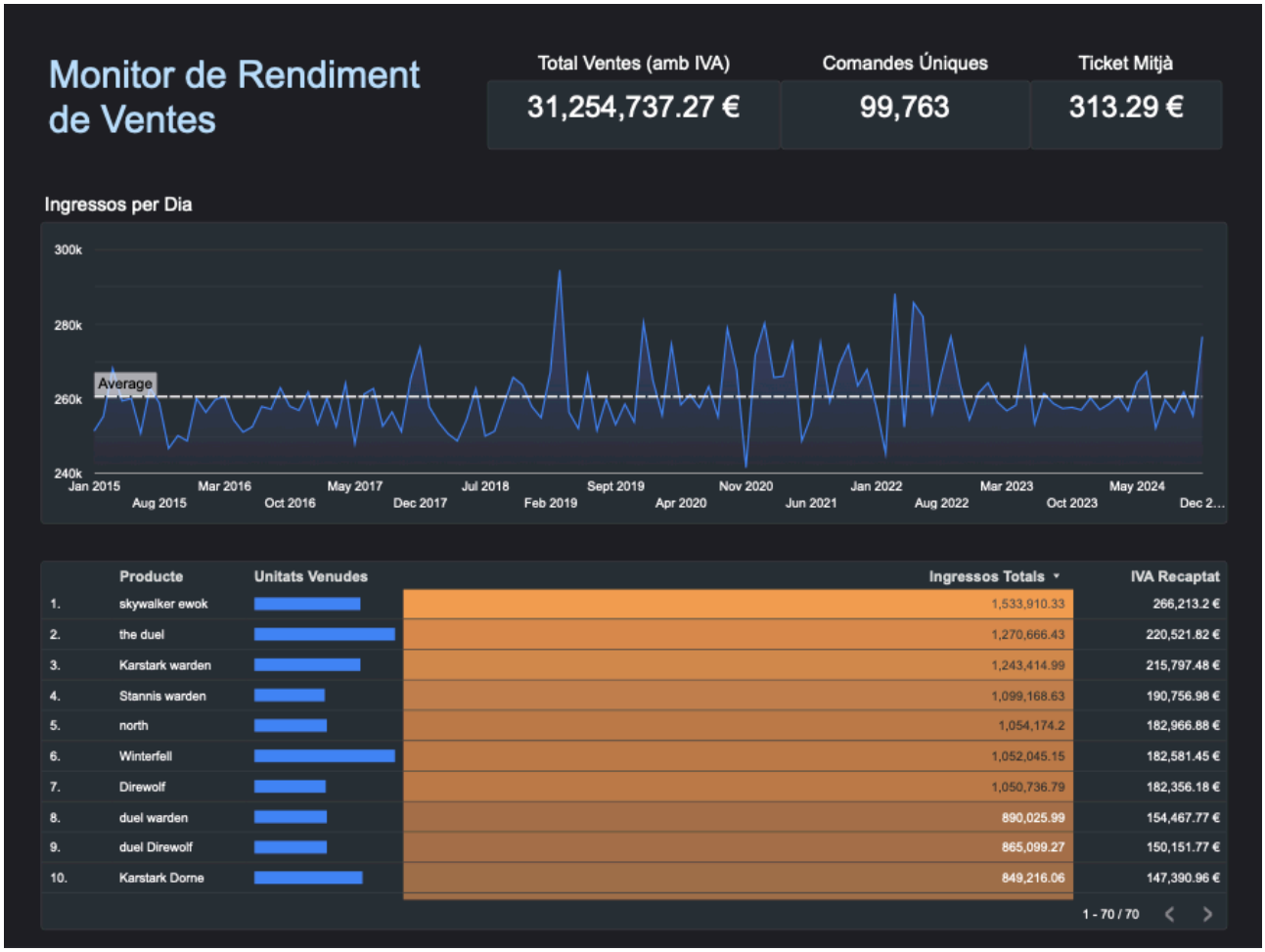
NOTA: una vez hechos los pasos para programar esta tarea, me da error al intentar guardarla porqué no tengo activado el billing en este proyecte que está en entorno sandbox…



3. Visualització (BI): Connecta Looker Studio a la teva taula dim_transactions_flat i crea un Dashboard: "Monitor de Rendiment de Vendes".

» Enllaç del gràfic a Looker Studio:
https://datastudio.google.com/s/iC5gx7TWXro

He conectado la tabla a Data Studio (Looker), y he podido componer un **dashboard** con todas las gráficas y métricas que se proponen en el ejercicio:



REVISIÓN (Irene):

1.4:
DATE(timestamp)
en lugar de TIMESTAMP_TRUNC(transaction_time, DAY)

2.1:
El resultado final es correcto. Todavía la manera con la que escribiste el código no refleja correctamente la pregunta y no es la versión más optimizada. La CTE debería incluir solo el Cantidad de Transacciones, la Cantidad de Transacciones (COUNT), el Ticket Medio (AVG), la Compra Máxima (MAX) y el filtro. Después se utiliza la CTE para hacer la JOIN con la tabla users_combines. Esto permite hacer la JOIN solo después de haber aplicado el filtro, entonces con menos datos. Este es uno de los valores de utilizar las CTEs.

2.3:
En este caso no es necesario utilizar una CTE.
Puedes directamente hacer una SELECT utilizando mv_daily_sales.

2.4:
El código es correcto, pero no está optimizado. Como vimos en el punto 1 de este nivel, una de las principales ventajas de una CTE es reducir la cantidad de datos con los que se trabaja antes de realizar una JOIN. Por lo tanto, idealmente, la consulta primero filtra los datos en una CTE, luego los agrupa y solo entonces realiza la JOIN. Ejemplo de consulta optimizada.